

Università degli Studi di Verona

Corso di laurea in Informatica (L-31)

**Progettazione e sviluppo
del frontend di un sistema
di gestione bibliotecaria**

Studente	Rusescu Mario Daniel
Matricola	VR523963

Indice

1	Introduzione	3
2	React: origine e principi fondamentali	3
2.1	Che cos'è React	3
2.2	Componenti, stato e rendering	4
2.3	Hook ed effetti	4
3	Tecnologie e strumenti adottati	5
3.1	TypeScript	6
3.2	Vite	6
3.3	Tailwind CSS e progettazione responsive	6
3.4	Axios	7
3.5	React Router	7
4	Backend e API REST	8
4.1	Ruolo del backend	8
4.2	Endpoint relativi ai libri	8
4.3	Endpoint relativi agli autori	8
5	Architettura del frontend	9
5.1	Organizzazione del codice	9
5.2	Provider globali	9
5.3	Flusso dei dati	9
6	Gestione dello stato remoto con TanStack Query	10
6.1	Query dichiarative	10
6.2	Mutation	10
6.3	Perché React Query è preferibile a una fetch in useEffect	11
7	Form e operazioni CRUD	11
7.1	TanStack Form	11
7.2	Creazione e selezione dell'autore	12
7.3	Modifica ed eliminazione	12
8	Visualizzazione con TanStack Table	12
9	Sfondo animato con WebGL	13
10	Recupero delle copertine dei libri	13
11	Avvio e test dell'applicazione	15
12	Conclusioni	15

1 Introduzione

Il progetto consiste nella realizzazione del frontend di un applicativo web per la gestione di una biblioteca. L'interfaccia permette di consultare e amministrare libri e autori ed eseguire operazioni CRUD.

Lo stack tecnologico è costituito da React e TypeScript, con Vite come ambiente di sviluppo e strumento di build. React Router gestisce la navigazione client-side, TanStack Query il recupero e la sincronizzazione dei dati remoti, TanStack Form lo stato dei moduli e TanStack Table la rappresentazione tabellare degli autori. Le richieste HTTP sono effettuate tramite Axios, mentre Tailwind CSS è impiegato per lo stile e la progettazione responsive.

Dal punto di vista funzionale, l'applicazione permette di:

- consultare l'elenco dei libri mediante una griglia responsive;
- cercare e paginare i risultati mantenendo i parametri nell'URL;
- visualizzare i dettagli di un libro in una finestra modale;
- creare, modificare ed eliminare libri;
- consultare gli autori in una tabella;
- creare, cercare, modificare ed eliminare autori;
- navigare tra le diverse sezioni senza ricaricare l'intera pagina.

La relazione presenta prima il contesto teorico delle tecnologie utilizzate, quindi descrive il backend fornito e infine analizza l'architettura e le principali scelte implementative del frontend.

2 React: origine e principi fondamentali

2.1 Che cos'è React

React è una libreria JavaScript per la costruzione di interfacce utente. È stata sviluppata internamente da Facebook, oggi Meta, e resa open source nel 2013. La documentazione ufficiale conserva lo storico delle prime versioni pubblicate a partire da maggio 2013.

React nasce per affrontare la crescente complessità delle interfacce web interattive. In un'applicazione tradizionale basata sulla manipolazione diretta del DOM, lo sviluppatore deve individuare manualmente gli elementi da aggiornare e coordinare tali modifiche con lo stato dell'applicazione. All'aumentare delle interazioni, questa sincronizzazione può diventare difficile da mantenere.

React adotta invece un modello dichiarativo: lo sviluppatore descrive quale interfaccia deve essere mostrata in funzione di dati e stato, mentre la libreria si occupa di aggiornare il DOM quando tali valori cambiano. L'interfaccia viene scomposta in componenti riutilizzabili, ciascuno responsabile di una parte della pagina.

Un componente funzionale può essere rappresentato nel modo seguente:

```
type MenuCardProps = {
  title: string;
  link: string;
};

function MenuCard({ title, link }: MenuCardProps) {
  return <Link to={link}>{title}</Link>;
}
```

Il componente riceve dati attraverso le *props* e restituisce JSX, una sintassi che consente di descrivere la struttura dell'interfaccia all'interno di JavaScript o TypeScript. Le props rendono il componente configurabile: la stessa implementazione può essere utilizzata per più destinazioni cambiando soltanto titolo e collegamento.

2.2 Componenti, stato e rendering

Lo stato rappresenta la memoria locale di un componente. Quando viene aggiornato tramite una funzione fornita da React, il componente viene renderizzato nuovamente. Lo stato è locale alla specifica istanza del componente. Nel progetto questo principio è utilizzato, ad esempio, per memorizzare il libro selezionato:

```
const [isOpen, setIsOpen] = useState<Book | null>(null);

<BookCard
  bookTitle={book.title}
  isbn={book.isbn}
  isbn13={book.isbn13}
  onClick={() => setIsOpen(book)}
/>

{isOpen && (
  <BookModal book={isOpen} onClose={() => setIsOpen(null)} />
)}
```

Lo stato `isOpen` ha due significati coerenti: `null` indica che nessun dettaglio è aperto, mentre un oggetto `Book` identifica il contenuto della modale. Il rendering condizionale evita di mantenere nel DOM una modale non necessaria.

Stato locale e stato remoto. React distingue tra stato locale dell'interfaccia e stato remoto. Il primo comprende valori come l'apertura di un menu o il testo temporaneo di un input; il secondo comprende dati provenienti dal server, che possono diventare obsoleti ed essere modificati indipendentemente dal client. Questa distinzione è centrale nella scelta di TanStack Query.

2.3 Hook ed effetti

Gli Hook permettono ai componenti funzionali di utilizzare funzionalità di React quali stato, contesto ed effetti. `useEffect` è uno strumento per sincronizzare un componente con un sistema esterno, per esempio un event listener, un timer o una libreria non controllata da React. Non deve essere interpretato come il meccanismo universale per qualsiasi elaborazione successiva al rendering.

È possibile eseguire una richiesta HTTP con `useEffect`, ma questa soluzione richiede di costruire manualmente gran parte dell'infrastruttura necessaria:

```

useEffect(() => {
  let ignore = false;
  setLoading(true);

  getBooks(page)
    .then((result) => {
      if (!ignore) setBooks(result.data);
    })
    .catch((error) => {
      if (!ignore) setError(error);
    })
    .finally(() => {
      if (!ignore) setLoading(false);
    });

  return () => {
    ignore = true;
  };
}, [page]);

```

Oltre alla richiesta, occorre gestire caricamento, errore, annullamento logico, dipendenze ed eventuali condizioni di competizione. Questa implementazione non offre automaticamente cache, deduplicazione, aggiornamento in background o invalidazione coordinata. Per questo motivo il progetto usa `useEffect` soltanto quando serve un effetto di sincronizzazione, mentre delega lo stato remoto a TanStack Query.

3 Tecnologie e strumenti adottati

Le versioni utilizzate nel progetto sono riportate nella tabella seguente. Per le librerie frontend i valori corrispondono alle versioni installate nel progetto.

Strumento o libreria	Versione
Node.js	22.22.2
npm	11.16.0
Bun	1.3.14
React	19.2.4
React DOM	19.2.4
React Router DOM	7.13.1
TypeScript	5.9.3
Vite	7.3.5
Tailwind CSS	4.2.1
Axios	1.13.6
TanStack Query	5.100.5
TanStack Form	1.29.1
TanStack Table	8.21.3
Lucide React	0.577.0
Three.js	0.170.0
Postprocessing	6.39.1

Strumento o libreria	Versione
use-debounce	10.1.1
ESLint	9.39.4

3.1 TypeScript

TypeScript estende JavaScript con un sistema di tipi statici. Il codice viene controllato durante lo sviluppo e trasformato in JavaScript eseguibile dal browser. I tipi non sostituiscono la validazione dei dati ricevuti dal server, ma permettono di descrivere il contratto atteso e individuare prima dell'esecuzione molti errori relativi a proprietà mancanti o valori incompatibili. Il linguaggio supporta tipi primitivi, oggetti, unioni, generici e inferenza.

Nel progetto i modelli restituiti dalle API sono descritti esplicitamente:

```
export type Book = {
  id: number;
  title: string;
  isbn?: string;
  authorId: number;
  authorName?: string;
  averageRating?: number;
  createdAt: string;
  updatedAt: string;
};

export type ApiResponse<T> = {
  data: T[];
  pagination: Pagination;
};
```

Il generico `ApiResponse<T>` evita di duplicare la struttura della risposta paginata. Le proprietà opzionali rappresentano campi che il backend può restituire senza valore. È stato inoltre separato il modello completo dai DTO utilizzati per creazione, aggiornamento e parametri di ricerca.

3.2 Vite

Vite è lo strumento di sviluppo e build utilizzato dal progetto. In sviluppo serve i moduli sorgente tramite ES Modules e applica l'Hot Module Replacement, aggiornando rapidamente nel browser i moduli modificati. Per la produzione crea invece asset statici ottimizzati. Vite comprende un development server e un comando di build per la produzione.

3.3 Tailwind CSS e progettazione responsive

Tailwind CSS adotta un approccio *utility-first*: invece di creare una classe CSS specifica per ogni elemento, si combinano classi atomiche direttamente nel markup. Ciò rende visibili nello stesso punto struttura, spaziatura, colore e comportamento responsive.

Un esempio è la griglia dei libri:

```
<div className="grid grid-cols-2 gap-x-3 gap-y-6
  md:grid-cols-3 xl:grid-cols-5">
  {books.map((book) => <BookCard key={book.id} /* ... */ />)}
</div>
```

La definizione è *mobile first*: in assenza di prefissi sono mostrate due colonne; da `md` le colonne diventano tre e da `xl` cinque. La singola copertina utilizza `aspect-[2/3]` per mantenere le proporzioni e dimensioni progressive ai diversi breakpoint.

3.4 Axios

Axios è utilizzato come client HTTP. Una sua istanza centralizza l'indirizzo base del server:

```
import axios from "axios";

export const api = axios.create({
  baseURL: "http://localhost:3000/",
});
```

Le funzioni del livello API isolano i componenti dai dettagli delle richieste:

```
export async function getPaginatedBooks(params: BooksPageDto) {
  return (
    await api.get("/books", {
      params: {
        page: params.page,
        limit: params.limit,
        search: params.search || undefined,
      },
    })
  ).data;
}
```

Questa separazione migliora la leggibilità: i componenti dichiarano quale operazione richiedono, mentre URL, metodo HTTP e trasformazione della risposta rimangono nel modulo api.

3.5 React Router

React Router implementa la navigazione client-side. BrowserRouter, collocato alla radice dell'applicazione, osserva la cronologia del browser; Routes associa invece i percorsi ai componenti.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/books" element={<AllBooks />} />
  <Route path="/booksmenu" element={<BooksLobby />} />
  <Route path="/create-book" element={<CreateBook />} />
  <Route path="/edit-book/:id" element={<EditBook />} />
  <Route path="/edit-author/:id" element={<EditAuthor />} />
  <Route path="/authors" element={<AllAuthors />} />
  <Route path="*" element={<Home />} />
</Routes>
```

I segmenti `:id` sono parametri dinamici, letti tramite `useParams`. `Link` consente la navigazione senza ricaricare il documento, mentre `useNavigate` permette spostamenti imperativi dopo operazioni quali l'eliminazione.

Ricerca e paginazione sono memorizzate nei query parameter tramite `useSearchParams`:

```
const [searchParams, setSearchParams] = useSearchParams({ page: "1" });
const page = Number(searchParams.get("page")) || 1;
const search = searchParams.get("search") ?? "";
```

Questa scelta rende lo stato della lista condivisibile e compatibile con i pulsanti avanti e indietro del browser. Un URL come `/books?page=1&search=libro` descrive direttamente la vista richiesta.

4 Backend e API REST

4.1 Ruolo del backend

Le risorse principali sono books e authors. Ogni libro contiene il riferimento all'autore tramite authorId; le risposte di lettura includono anche authorName per evitare al client una seconda richiesta nella visualizzazione ordinaria.

4.2 Endpoint relativi ai libri

Metodo	Percorso	Funzione
GET	/books	Restituisce una lista paginata. Supporta ricerca per titolo o autore e filtri per autore, lingua, valutazione e editore.
GET	/books/:id	Restituisce il dettaglio di un libro con le informazioni dell'autore.
POST	/books	Crea un libro. title e authorId sono obbligatori; l'autore deve esistere.
PUT	/books/:id	Aggiorna uno o più campi di un libro esistente.
DELETE	/books/:id	Elimina il libro identificato dall'ID.

La lista accetta i parametri page, limit, search, authorId, languageCode, minRating, maxRating e publisher. Il frontend corrente utilizza principalmente pagina, limite e ricerca. La risposta ha la seguente forma concettuale:

```
{
  "data": [
    {
      "id": 1,
      "title": "Titolo",
      "authorId": 10,
      "authorName": "Nome autore"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 100,
    "total": 250,
    "totalPages": 3
  }
}
```

Il server valida i parametri numerici e restituisce 400 Bad Request quando pagina, limite o valutazioni non sono validi. Per le operazioni su un'entità inesistente viene restituito 404 Not Found; la creazione corretta utilizza lo stato 201 Created.

4.3 Endpoint relativi agli autori

Metodo	Percorso	Funzione
GET	/authors	Restituisce una lista paginata e permette la ricerca per nome.
GET	/authors/:id	Restituisce il dettaglio dell'autore richiesto.
GET	/authors/:id/books	Restituisce tutti i libri associati a un autore.
POST	/authors	Crea un autore; il nome è obbligatorio.

Metodo	Percorso	Funzione
PUT	/authors/:id	Aggiorna il nome dell'autore.
DELETE	/authors/:id	Elimina un autore soltanto se non possiede libri associati.

5 Architettura del frontend

5.1 Organizzazione del codice

Il frontend è organizzato per responsabilità:

```
src/
├─ api/           # client Axios e funzioni HTTP
├─ components/    # componenti riutilizzabili
├─ pages/         # componenti associati alle route
├─ types/         # modelli TypeScript e DTO
├─ App.tsx       # configurazione delle route
├─ main.tsx      # bootstrap e provider globali
└─ index.css     # Tailwind e stili globali
```

Le pagine coordinano il recupero dei dati e la navigazione. I moduli API fungono da confine tra UI e server. Questa suddivisione riduce l'accoppiamento e consente di modificare un livello senza riscrivere l'intera applicazione.

5.2 Provider globali

Il punto di ingresso crea la radice React e installa i provider necessari:

```
const queryClient = new QueryClient();

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </QueryClientProvider>
  </StrictMode>,
);
```

QueryClientProvider rende disponibile la cache di TanStack Query a tutti i componenti discendenti. BrowserRouter fornisce il contesto di navigazione.

5.3 Flusso dei dati

Il flusso principale può essere riassunto in cinque passaggi:

1. la route determina quale pagina deve essere renderizzata;
2. la pagina legge parametri dinamici o query parameter dall'URL;
3. useQuery richiama una funzione del livello API;
4. Axios invia la richiesta al backend e restituisce i dati JSON tipizzati;
5. React aggiorna dichiarativamente la vista quando lo stato della query cambia.

Le operazioni di scrittura seguono un flusso analogo mediante useMutation: il form raccoglie i dati, la mutation invia POST, PUT o DELETE, quindi l'interfaccia comunica l'esito ed eventualmente cambia route.

6 Gestione dello stato remoto con TanStack Query

6.1 Query dichiarative

TanStack Query, precedentemente denominata React Query, è una libreria per recuperare, memorizzare in cache, sincronizzare e aggiornare lo stato remoto. Nel progetto una lista di libri viene richiesta nel modo seguente:

```
const { data, isLoading } = useQuery({
  queryKey: ["books", page, search],
  queryFn: () =>
    getPaginatedBooks({
      page,
      limit: 100,
      search: search || undefined,
    }),
});

const books = data?.data ?? [];
```

La queryKey identifica logicamente i dati. Pagina e ricerca ne fanno parte, quindi ogni combinazione possiede una voce di cache distinta. Quando cambia l'URL, cambiano page o search; la chiave viene aggiornata e TanStack Query esegue la funzione appropriata.

Per la ricerca dell'autore durante la creazione di un libro è utilizzata una query condizionale:

```
const [debouncedSearch] = useDebounce(authorSearch, 300);

const { data: authorsData } = useQuery({
  queryKey: ["authors", debouncedSearch],
  queryFn: () => getAllAuthors({ search: debouncedSearch, limit: 10 }),
  enabled: !!debouncedSearch,
});
```

Il debounce attende 300 millisecondi dopo l'ultima digitazione, riducendo il numero di richieste. L'opzione enabled impedisce l'esecuzione con una ricerca vuota.

6.2 Mutation

Le operazioni che modificano lo stato remoto sono modellate mediante useMutation:

```
const deleteMutation = useMutation({
  mutationFn: () => deleteBook(book.id),
  onSuccess: () => {
    alert("Libro eliminato.");
    navigate("/booksmenu");
  },
  onError: () => {
    alert("Non siamo riusciti a eliminare il libro.");
  },
});
```

La mutation espone anche informazioni quali isPending, utilizzate per disabilitare i pulsanti durante l'operazione. Questo evita invii ripetuti accidentali.

6.3 Perché React Query è preferibile a una fetch in useEffect

Il vantaggio non consiste semplicemente in una sintassi più breve. `useEffect` e `TanStack Query` risolvono problemi differenti: il primo sincronizza il componente con sistemi esterni, mentre il secondo modella esplicitamente il ciclo di vita dello stato remoto.

Aspetto	Richiesta manuale in <code>useEffect</code>	<code>TanStack Query</code>
Caricamento ed errore	Stati e transizioni implementati manualmente.	Stati <code>isLoading</code> , <code>isError</code> , <code>isFetching</code> già disponibili.
Cache	Da progettare e mantenere separatamente.	Cache associata alla <code>queryKey</code> .
Richieste duplicate	Possibili se più componenti richiedono gli stessi dati.	Coordinamento e deduplicazione tramite cache.
Dati obsoleti	Richiede regole personalizzate.	Concetti espliciti di dati freschi e stale.
Aggiornamento	Dipendenze ed effetti aggiuntivi.	<code>Refetch</code> e invalidazione dichiarativi.
Race condition	<code>Cleanup</code> e annullamento a carico dello sviluppatore.	Ciclo di vita gestito dalla libreria.
Manutenibilità	Logica distribuita tra più state ed effect.	Configurazione concentrata in <code>query</code> e <code>mutation</code> .

Lo stato remoto è asincrono, può cambiare fuori dal controllo del client e introduce problemi di cache, aggiornamento e deduplicazione. `React Query` gestisce questi aspetti in un unico punto, evitando di ripetere in ogni componente la stessa logica per richieste, caricamento ed errori.

7 Form e operazioni CRUD

7.1 TanStack Form

I form di creazione e modifica utilizzano `@tanstack/react-form`. `useForm` definisce i valori iniziali e la funzione di submit; `form.Field` collega ogni input al proprio stato.

```
const form = useForm({
  defaultValues: {
    title: "",
    authorId: 0,
    publisher: "",
    isbn: "",
  },
  onSubmit: async ({ value }) => {
    createMutation.mutate(value);
  },
});
```

```
<form.Field
  name="title"
  children={({field}) => (
    <input
      name={field.name}
      value={field.state.value}
      onChange={(event) => field.handleChange(event.target.value)}
    />
  )}
/>
```

Il campo è controllato: il valore mostrato proviene dallo stato del form e ogni modifica viene comunicata attraverso `handleChange`. Per i campi numerici viene applicata una conversione esplicita con `Number`, poiché il valore di un input HTML è sempre una stringa.

`form.Subscribe` osserva soltanto la porzione necessaria dello stato e controlla la disponibilità del pulsante:

```
<form.Subscribe
  selector={({state}) => [state.canSubmit, state.isSubmitting]}
  children=(([canSubmit, isSubmitting]) => (
    <button disabled={!canSubmit || isSubmitting}>Save</button>
  ))
/>
```

7.2 Creazione e selezione dell'autore

La creazione di un libro comprende numerosi campi del dominio: titolo, autore, editore, data, numero di pagine, lingua, ISBN e valori relativi alle valutazioni. La selezione dell'autore è assistita da una ricerca asincrona. Quando l'utente sceglie un risultato, il testo mostra il nome ma nel form viene memorizzato l'identificativo numerico richiesto dall'API.

7.3 Modifica ed eliminazione

La pagina di modifica legge l'ID dalla route, recupera il record con `useQuery` e inizializza il form soltanto quando il libro è disponibile. Aggiornamento ed eliminazione sono due mutation separate. La stessa impostazione è stata adottata per gli autori; l'errore restituito dal backend in caso di autore ancora collegato a libri viene presentato all'utente.

8 Visualizzazione con TanStack Table

TanStack Table è una libreria *headless*: fornisce modelli e funzioni per costruire tabelle, ma non impone il markup o lo stile. Questa caratteristica consente di integrare la logica della tabella con Tailwind.

Le colonne degli autori sono definite separatamente dai dati. `useReactTable` costruisce il modello delle righe:

```
const table = useReactTable({
  data: authors,
  columns,
  getCoreRowModel: getCoreRowModel(),
});
```

Il rendering utilizza `flexRender`, che gestisce sia contenuti testuali sia funzioni di rendering:

```
{table.getRowModel().rows.map((row) => (
  <tr key={row.id}>
    {row.getVisibleCells().map((cell) => (
      <td key={cell.id}>
        {flexRender(cell.column.columnDef.cell, cell.getContext())}
      </td>
    ))}
  </tr>
))}
```

La paginazione rimane server-side: cambiando pagina viene aggiornata la query string e useQuery richiede il nuovo insieme di autori. TanStack Table si occupa quindi della rappresentazione dei record già ricevuti, non del loro caricamento.

9 Sfondo animato con WebGL

Lo sfondo animato dell'applicazione è realizzato tramite il componente `PixelBlast`, inserito nel file `src/components/PixelBlast.tsx`. Il componente è stato preso da React Bits, una raccolta di componenti animati per React. Il riferimento utilizzato è disponibile all'indirizzo <https://www.reactbits.dev/backgrounds/pixel-blast>.

`PixelBlast` utilizza WebGL tramite `Three.js` e la libreria `postprocessing`. Nel progetto non è stato trattato come logica di dominio, ma come componente grafico riutilizzabile: viene importato nelle pagine che richiedono lo sfondo e posizionato in un contenitore `fixed` dietro al contenuto principale.

Un esempio di utilizzo è il seguente:

```
<div className="fixed inset-0 z-0">
  <PixelBlast
    variant="square"
    pixelSize={4}
    color="#B497CF"
    patternScale={2}
    patternDensity={1}
    enableRipples
    rippleSpeed={0.3}
    rippleThickness={0.1}
    rippleIntensityScale={1}
    speed={0.5}
    transparent
    edgeFade={0.25}
  />
</div>
```

Il contenuto della pagina viene invece collocato sopra lo sfondo con `z-10`. In questo modo l'animazione rimane separata dalla struttura dell'interfaccia: il componente gestisce il canvas WebGL e i suoi parametri grafici, mentre le pagine decidono solo dove mostrarlo e con quali proprietà configurarlo.

10 Recupero delle copertine dei libri

I dati restituiti dal backend contengono i codici ISBN e ISBN-13, ma non includono direttamente un'immagine di copertina. Per ottenere una copertina è stata quindi creata la funzione `getBookCoverUrl`, contenuta nel file `src/utls/image.ts`. La funzione costruisce l'indirizzo dell'immagine utilizzando il servizio `Open Library Covers`.

```

export const FALLBACK_BOOK_COVER_URL =
  "https://cdn.europosters.eu/pod_public/1300/138617.jpg";

export function getBookCoverUrl(isbn?: string, isbn13?: string) {
  const normalizedIsbn = isbn?.trim();
  const normalizedIsbn13 = isbn13?.trim();

  if (normalizedIsbn && normalizedIsbn !== "not found") {
    return `https://covers.openlibrary.org/b/isbn/${normalizedIsbn}-L.jpg?
default=false`;
  }
  if (normalizedIsbn13 && normalizedIsbn13 !== "not found") {
    return `https://covers.openlibrary.org/b/isbn/${normalizedIsbn13}-L.jpg?
default=false`;
  }
  return FALLBACK_BOOK_COVER_URL;
}

```

La funzione riceve i due codici come parametri opzionali. Se è disponibile l'ISBN, viene inserito nell'URL richiesto da Open Library; in caso contrario viene utilizzato l'ISBN-13. Il suffisso -L.jpg richiede la versione grande della copertina, mentre il parametro default=false evita che il servizio restituisca automaticamente un'immagine generica quando il codice non è associato a una copertina.

La funzione riceve i due codici come parametri opzionali. Se è disponibile l'ISBN, viene inserito nell'URL richiesto da Open Library; in caso contrario viene utilizzato l'ISBN-13. Il suffisso -L.jpg richiede la versione grande della copertina, mentre il parametro default=false evita che il servizio restituisca automaticamente un'immagine generica quando il codice non è associato a una copertina. Se nessun codice è disponibile, viene restituita direttamente una copertina di fallback, evitando richieste inutili verso Open Library.

Nella card del libro l'indirizzo viene calcolato quando il componente viene inizializzato e salvato nello stato locale. Poiché il download delle immagini esterne può essere lento, è stato aggiunto anche uno stato dedicato al caricamento:

```

const [imgSrc, setImgSrc] = useState(getBookCoverUrl(isbn, isbn13));
const [isImageLoading, setIsImageLoading] = useState(true);

<img
  src={imgSrc}
  alt={bookTitle}
  onLoad={() => setIsImageLoading(false)}
  onError={() => {
    setIsImageLoading(true);
    setImgSrc(FALLBACK_BOOK_COVER_URL);
  }}
/>

```

L'attributo src riceve quindi l'URL generato. Quando l'immagine termina il caricamento, onLoad imposta isImageLoading a false; fino a quel momento la card mostra uno skeleton animato con animate-pulse. Se la richiesta non produce un'immagine valida, l'evento onError sostituisce l'indirizzo con la copertina di fallback. Lo skeleton rimane visibile finché non viene caricata anche l'immagine sostitutiva.

```
{isLoading && (  
  <div className="absolute inset-0 z-10 animate-pulse bg-gradient-to-br  
    from-[#2a2040] via-[#3b2a58] to-[#1e1e2e]" />  
)}
```

Questo meccanismo evita che l'utente veda spazi vuoti durante il caricamento delle copertine. Inoltre l'overlay con il titolo del libro è posizionato sopra lo skeleton, così il nome resta visibile al passaggio del mouse anche se l'immagine non è ancora pronta.

11 Avvio e test dell'applicazione

Per verificare il funzionamento del progetto è necessario avviare separatamente backend e frontend. Il backend deve essere eseguito per primo, perché il frontend recupera libri e autori tramite richieste HTTP verso le API esposte dal server.

Nella cartella del backend si esegue:

```
bun run dev
```

Il comando usa lo script dev definito nel backend ed esegue il server in modalità sviluppo. In alternativa è possibile avviare il server senza modalità watch:

```
bun run start
```

Una volta avviato il backend, in un secondo terminale, dalla cartella del frontend, il primo comando da eseguire è:

```
npm install
```

Questo comando installa le dipendenze definite in `package.json` e bloccate in `package-lock.json`. Dopo l'installazione si avvia il frontend con:

```
npm run dev
```

Il comando avvia il development server di Vite. Aprendo nel browser l'indirizzo indicato dal terminale è possibile navigare nell'applicazione e verificare le funzionalità principali: caricamento dei libri, ricerca, paginazione, visualizzazione dei dettagli, creazione, modifica ed eliminazione delle entità gestite.

Il test manuale richiede quindi che entrambi i processi rimangano attivi: il backend fornisce i dati tramite API REST, mentre il frontend mostra l'interfaccia React e invia le richieste tramite Axios e TanStack Query.

12 Conclusioni

Il progetto ha permesso di realizzare un frontend completo per la gestione di una biblioteca. React ha fornito il modello a componenti e il rendering dichiarativo; React Router ha collegato lo stato della navigazione agli URL; TanStack Query ha modellato in modo esplicito il ciclo di vita dei dati remoti; TanStack Form e TanStack Table hanno organizzato rispettivamente moduli e rappresentazione tabellare. TypeScript ha descritto i contratti dei dati, Axios ha isolato la comunicazione HTTP, Tailwind CSS ha supportato la costruzione di un'interfaccia responsive e Vite ha fornito l'ambiente di sviluppo e build.

Il confronto con una gestione delle richieste basata esclusivamente su `useEffect` evidenzia il principale risultato metodologico dell'attività: lo stato proveniente dal server non è soltanto un valore da caricare

una volta, ma possiede identità, validità temporale, stati asincroni e necessità di sincronizzazione. Affidare questi aspetti a una libreria specializzata riduce il codice accidentale e rende più esplicito il comportamento dell'applicazione.

L'esperienza mostra inoltre come le librerie adottate non sostituiscano la progettazione: query key, confini tra componenti, tipi, URL e responsabilità dei moduli devono comunque essere definiti coerentemente. Il risultato è un'applicazione navigabile e responsive, capace di eseguire le principali operazioni CRUD su libri e autori.